Liver Segmentation Using U-net

Aina Fernández González

1 Introduction

Medical image segmentation is crucial for diagnosis and treatment planning. In particular, in achieving precise hepatic segmentation from computed tomography (CT) imaging scans is vital for assessing diseases, planning surgeries, and analyzing volumes. However, manual segmentation by medical experts is slow and may vary between different observers, which motivates the development of automated methods.

In recent years, deep learning techniques, especially convolutional neural networks (CNNs), have significantly improved performance in many computer vision tasks. Unlike traditional image processing methods that depend on manually crafted features, CNNs can learn hierarchal features directly from data. This ability has made them particularly suitable for analyzing biomedical images, where the aim is often to assign a class label to every pixel in an image.

Among the proposed architectures, the U-Net model has become one of the most popular choices for biomedical image segmentation. U-Net has an encoder-decoder structure that captures contextual information through a contracting path and restores spatial resolution through an expanding path. Skip connections between matching layers allow the network to blend high-level semantic information with detailed spatial data, enabling precise pixel-level predictions even with limited training data.

In this project, a U-Net based approach is used to perform automatic liver segmentation on medical images. The goal is to train and test the model on a liver imaging dataset and evaluate its performance in terms of segmentation accuracy. The results offer insights into the effectiveness of deep learning methods for medical image segmentation tasks.

2 State of the art

Liver segmentation in medical imaging, particularly in CT scans, is crucial for diagnosis, treatment planning, and monitoring diseases. Over time, methods have changed from manual to fully automated deep learning approaches [4].

Early methods depended on manual segmentation, where experts outlined liver boundaries. While this method is accurate, it is slow and can vary between different observers. To reduce this workload, semi-automatic methods were introduced, which combine algorithmic help with human input.

Classical automatic methods include threshold-based segmentation, which separates tissues using intensity values but is sensitive to noise. Region-growing methods expand from seed points based on similarity criteria, but the results rely heavily on seed selection. Edge-based methods find boundaries using gradients, but they can be unreliable in noisy or low-contrast images. Shape-based models use anatomical information, which improves reliability but adds to computational complexity[4].

Recently, deep learning methods, especially convolutional neural networks (CNNs), have significantly improved segmentation performance [5]. These models learn features directly from the data, cutting down the need for manually crafted features. Among these, U-Net is one of the most popular architectures because of its encoder-decoder structure and skip connections, which effectively combine contextual and spatial information [3].

Overall, CNN-based approaches have become the main solution for liver segmentation, surpassing traditional methods in accuracy and reliability. U-Net and its variations are the standard reference in most modern medical image segmentation systems.

3 Methodology

3.1 Dataset

The dataset consists of 400 PNG images representing 2D slices extracted from abdominal CT scans. Each image is paired with a corresponding segmentation mask indicating the liver region. To facilitate data loading during training, a custom PyTorch dataset class was implemented to manage the images and their corresponding masks.

The dataset was divided into training and validation subsets, containing 360 images for training and 40 images for validation.

3.2 Preprocessing

The original images and masks have a resolution of 512×512 pixels. To reduce computational cost and improve training efficiency, both were resized to 256×256 pixels. The images were then converted into PyTorch tensors before being used as input to the model.

Segmentation masks were binarized so that each pixel represents one of two classes: background or liver.

3.3 Model Architecture

The segmentation model is based on the U-Net architecture proposed by Ronneberger et al. [3]. The network follows the original encoder–decoder design composed of a contracting path and an expansive path.

The encoder consists of repeated blocks of 3×3 convolutions with ReLU activations followed by 2×2 max-pooling operations for downsampling.

The decoder performs upsampling through transposed convolutions and concatenates the resulting feature maps with the corresponding encoder feature maps via skip connections. This allows the network to recover spatial information lost during downsampling.

The final layer applies a 1×1 convolution to map the learned features to a two-channel output corresponding to the background and liver classes.

3.4 Training Setup

The model was trained using an Adam optimizer with a learning rate of 1e-3, a common configuration that pro-

vides a good balance between convergence speed and training stability. The loss function used during training was *CrossEntropyLoss*, which is suitable for pixel classification problems.

The model was trained for 50 epochs, which was considered sufficient to observe the learning behavior while keeping the training time manageable.

Training was implemented in PyTorch and executed on the available hardware resources.

3.5 Evaluation Metrics

The performance of the model was evaluated using mean Intersection over Union (mIoU) and mean Pixel Accuracy (mPA), which are standard metrics for semantic segmentation tasks.

mIoU measures the overlap between the predicted segmentation and the ground truth, providing an overall indication of segmentation quality across classes. mPA evaluates the proportion of correctly classified pixels with respect to the total number of pixels.

4 IMPLEMENTATION

The implemented system is built in PyTorch and consists of a complete pipeline for medical image segmentation, including data processing, training, evaluation, and reporting. The core model is based on the U-Net architecture proposed by Ronneberger et al. [3], re-implemented for this project.

4.1 Software Framework and Project Structure

The system was built in Python using the PyTorch deep learning framework. I chose PyTorch for its flexibility, extensive support for neural network development, and its dynamic computation graph.

Several supporting libraries were also used to facilitate different parts of the project. For example for image handling and loading I used the Python Imaging Library (PIL), which allows efficient manipulation of PNG image files before they are converted into tensors. Data transformations and tensor conversion were also implemented by using the *torchvision.transforms* module. Other additional utilities such as *tqdm* were used to provide progress bars during training.

The project is divided into several modules in order to separate responsibilities across the pipeline. Some of these files correspond to the main pipeline, such as the model definition, dataset handling, training pipeline, and report generation.

The *main* script acts as the entry point of the project and coordinates the execution flow through a simple command-line interface. It connects the different modules and manages the overall pipeline execution.

Additional features are organized into separate auxiliary modules. For instance, the *utils* module contains helper functions related to evaluation and other shared operations. To ensure reliable execution regardless of the working directory, a *paths* module was implemented. This module centralizes all relevant project paths, reducing potential path-related errors.

This modular design facilitates debugging and allows different components of the system to be reused or extended independently, as shown in Figure 1.

```
project/
├── src/
│   ├── main.py
│   ├── train.py
│   ├── unet.py
│   ├── dataset.py
│   ├── utils.py
│   ├── report.py
│   └── paths.py
├── data/
│   ├── Images/
│   └── Labels/
├── models/
│   ├── custom_baseline.pth
│   ├── default_pretrained.pth
│   └── additional trained models
├── reports/
│   ├── default_report.txt
│   └── additional reports
├── README.md
└── requirements.txt
```

Fig. 1: Project directory structure.

4.2 Dataset Loading

The dataset loading pipeline is implemented through a custom class named *LiverDataset*, it's based on PyTorch's *Dataset* class. Its purpose is to link input images with their corresponding segmentation masks while providing compatibility with PyTorch *DataLoader* objects via tensors.

The preprocessing pipeline is defined during initialization, it includes image resizing and tensor conversion to work with the model using *torchvision.transforms*.

4.3 Model (U-Net)

As mentioned in Section 3.3, a basic U-Net segmentation module using PyTorch is implemented. The model follows an encoder–decoder structure with skip connections.The final layer uses a 1×1 convolution to generate a classification map for the segmentation task.

The implementation of the U-Net architecture was not developed from scratch. The original structure was adapted from an external source and integrated into the project.

4.4 Training Pipeline

The training module encapsulates the main functional component of the program in a *train()* function. It's responsible for optimizing the U-net module using the training dataset and evaluating its performance after each epoch.

During training, the model predictions are compared with the ground truth masks using the CrossEntropyLoss function. This loss measures the difference between the predicted classes and the correct labels. Backpropagation is then applied to compute how each parameter contributes

to the error, allowing the network to learn from its mistakes. Finally, the Adam optimizer updates the model weights using these gradients, gradually improving the model's performance over time. Additionally, in automatic mode, a learning rate scheduler (ReduceLROnPlateau) is used to dynamically adjust the learning rate at the end of each epoch. As shown in Listing 1.

```
criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=learning_rate
    )
scheduler = torch.optim.lr_scheduler.
    ReduceLROnPlateau(
    optimizer,
    mode="max",
    factor=0.5,
    patience=2
    ) if auto else None
```

Listing 1: Training configuration used during optimization.

Training is performed over multiple epochs. In each epoch, the model goes through a training phase followed by a validation phase. During training, the model wheights are updated based on the computed loss, while the validation phase is used to evaluate its performance using the validation set. The validation procedure, including the *evaluate()* function and the metric computation modules for Intersection over Union (IoU) and Pixel Accuracy, is implemented in a separate module named *utils.py*. As described in Section 4.6, the best model is then stored based on mIoU.

Additionally, an early stopping strategy is applied in automatic mode. If no improvement in mIoU is observed over a number of epochs, 5 epochs in this program, training is stopped early to avoid unnecessary computation.

4.5 Evaluation Pipeline

The *evaluate()* function returns two evaluation metrics, Intersection over Union (IoU) and Pixel Accuracy, used to quantify information about the model's performance.

IoU measures the overlap, or ratio between intersection and union, between the predicted segmentation and the ground truth, determining how well they match.

Pixel Accuracy determines the proportion of correctly classified pixels over the total number of pixels, providing a more global measure. Even so, Pixel Accuracy can be misleading as a model may achieve high accuracy by mostly classifying background pixels correctly. For this reason, I used IoU as the primary metric throughout training.

The evaluation process is implemented through a simple function that applies these metrics over the validation dataset. The *evaluate()* function calls both IoU and PA to obtain the final performance scores.

4.6 Checkpointing

A checkpointing mechanism is implemented using a diccionary to save relevant parameters of the model during training for later analysis.

For instance, to store the best model, the mIoU obtained after each epoch is used as the selection criterion. When the current mIoU exceeds the best recorded value, the model state is saved.

In automatic mode, the checkpoint also stores the current epoch and learning rate when the model improves. Additionally, the training loss is recorded after each epoch for later analysis.

```
checkpoint = {
    "model_state_dict": None,
    "epochs": epochs,
    "batch_size": batch_size,
    "learning_rate": learning_rate,
    "train_losses": []
}
```

Listing 2: Checkpoint initialization

4.7 Reporting System

The reporting system evaluates trained models and generates a structured summary of their performance.

In addition to storing training results, the model is re-evaluated using the entire dataset, as no additional test data is available. This provides a clear view of the model's overall performance using mean Intersection over Union (mIoU) and mean Pixel Accuracy (mPA).

Apart from numerical evaluation, the function includes a visualization module for qualitative analysis. It allows the inspection of individual segmentations, including the best, worst, and randomly selected cases based on IoU. Figure 2 shows an example of the best performing segmentation in a trained model. Other comparative visualizations such as the correlation between IoU and Pixel Accuracy, as well as the training loss evolution over epochs, are available for analysis.

Finally, the system generates a textual report (.txt file) that compiles all relevant information about the experiment. This includes dataset details, hyperparameters, training statistics, validation performance, and the best and worst segmentation results. The goal of this report is to summarize each trained model in a clear and reproducible way.

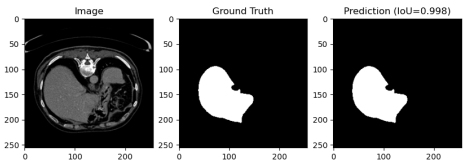


Fig. 2: Best performing segmentation based on IoU of an automatic pre-trained model

4.8 Limitations

The proposed system has some limitations. The dataset size is relatively small, which affects the model's ability to generalize data. In addition, no separate test set is used, as the same validation data is reused for final evaluation.

Finally, the model is constrained by available computational resources, limiting the complexity of the architecture and the training duration. As a result, the system is designed to run locally.

5 Results

This section shows a comparison between multiple training configurations used to evaluate the impact of different hyperparameters on the performance. The automatic configuration dynamically adjusts several training parameters during execution. For this particular model, the values reported in the table correspond to its final state. The learning rate, for example, starts at a default value of 1e-3 and is adapted throughout training. Batch size is determined during initialization based on the available hardware resources.

Model	Epochs	Batch Size	Learning Rate	mIoU
Automatic	67	1	3.9e-06	0.9927
Custom 1	50	1	1e-3	0.9533
Custom 2	50	2	1e-3	0.9849
Custom 3	25	1	1e-3	0.9792
Custom 4	50	1	1e-4	0.9828

Taula 1: Comparison of different training configurations and their performance.

The results show that the automatic configuration achieves the best overall performance in terms of mIoU. This confirms that the adaptive adjustment of hyperparameters, particularly the learning rate, improves convergence and segmentation quality.

Among the custom configurations, the 50-epoch model with batch size 2 achieves the best performance, indicating that increasing batch size can improve training stability. The 25-epoch configuration has a better performance than the *Custom 1* configuration, showing that longer training does not necessarily lead to better results.

Finally, the configuration with a reduced learning rate (1e-4) achieves intermediate performance, below the best models but still comparable to the rest.

6 Discussion on Intelligence

6.1 Intelligent Behavior

The system can be considered intelligent because it is capable of learning meaningful representations from medical image data and generating automatic liver segmentations without someone explicitly writing the instructions.

The systems ability to learn is reflected in the reduction of training loss and the improvement of mIoU during training. Additionally, it is reflected while trying different model configurations how they affect the model's performance.

Figure 3 shows the evolution of the training loss in the automatic configuration model. The progressive decrease in loss indicates that the model successfully learns relevant patterns from the training data.

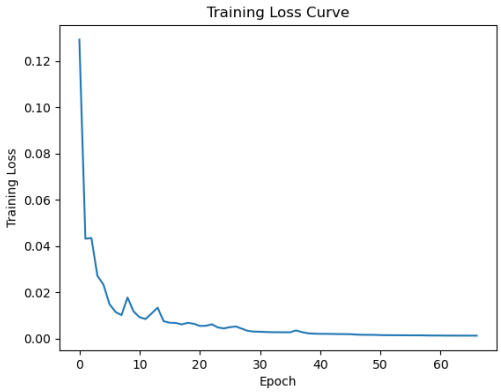


Fig. 3: Training loss evolution during training.

6.2 Quotations

AI-based tools were used as supporting aids during the development of this project. GitHub Copilot and Codex-based assistants were used to support writing, debugging, and optimization of the code. The AI tool integrated in Overleaf was used to improve writing quality and correct minor linguistic issues. ChatGPT was used to assist with the organisation and structuring of ideas throughout the project.

These tools were used to support the development process but did not replace the actual design, implementation, or validation work.

In addition, the U-Net architecture implementation was adapted from external online resources.

7 Conclutions

This project demonstrates the development of a complete deep learning pipeline for liver segmentation using a U-Net-based architecture. The system integrates dataset handling, model training, evaluation, and reporting.

The experimental results show that the model is able to learn meaningful representations from medical image data, achieving high performance in terms of mean Intersection over Union (mIoU) and mean Pixel Accuracy (mPA). The comparison between different training configurations highlights the importance of hyperparameter selection, particularly the learning rate and batch size, in model performance.

The automatic training configuration achieved the best overall results, showing that adaptive optimization strategies can improve the final accuracy.

Overall, the project successfully achieves its objectives by providing a complete system for training, evaluating, and analysing a medical image segmentation model in a reproducible way.

References

[1] LaTeX. (21 d'Abril de 2021). A Wikibooks. http://en.wikibooks.org/wiki/LaTeX

[2] O. Russakovskyet al., *ImageNet large scale visual recognition challenge*, Int. J. Comput. Vis., vol. 115, no. 3, pp. 211–252, 2015.

[3] O. Ronneberger, P. Fischer, and T. Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, arXiv:1505.04597, 2015.

[4] Wei, D., Jiang, Y., Zhou, X., Wu, D., Feng, X. (2024). *A Review of Advancements and Challenges in Liver Segmentation. Journal of Imaging*, 10(8), 202. https://doi.org/10.3390/jimaging10080202

[5] Litjens, G., Kooi, T., Bejnordi, B. E., Setio, A. A. A., Ciompi, F., Ghafoorian, M., van der Laak, J. A. W. M., van Ginneken, B., Sánchez, C. I. (2017). *A survey on deep learning in medical image analysis. Medical Image Analysis*, 42, 60–88. https://doi.org/10.1016/j.media.2017.07.005